

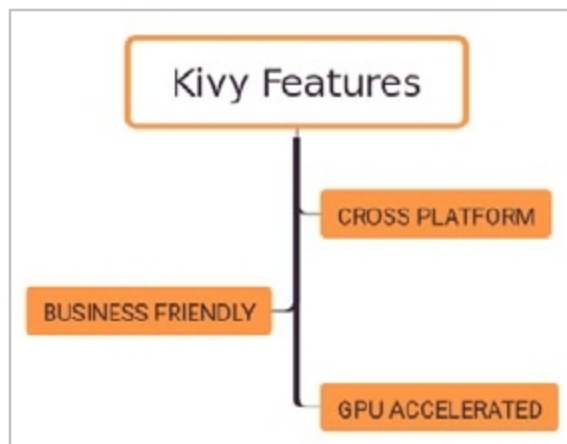


Figure 3: Kivy's features

- **Cross-platform:** Kivy supports the following platforms: Android, iOS, Raspberry Pi, Linux, OS X and Windows. The same code can be executed in all the supported platforms, which is a very useful feature for developers.
- **Business-friendly:** Kivy has no hidden costs. It is available with the MIT licence. Kivy is a stable framework and the documentation of the API is very good. Getting started with it is simple if you go through the official documentation.
- **GPU accelerated:** OpenGL ES2 is adapted to build the graphics engine.

Kivy installation

Kivy can be installed on various platforms. Detailed installation instructions are available at <https://kivy.org/#download>. For example, if you are using *conda*, it can be done simply with the following command:

```
$ conda install kivy -c conda-forge
```

'Hello World' app in Kivy

Building a 'Hello World' application involves the following steps.

- **Step 1:** Create a derived class for the app class.
- **Step 2:** Implement the build method.
- **Step 3:** Create an instance of this class.
- **Step 4:** Call the *run()* method. Sample code (from the official documentation) involving the above

steps is shown below:

```
import kivy

kivy.require('1.0.6') # Use you kivy
version here.

from kivy.app import App
from kivy.uix.label import Label

class MyApp(App):

    def build(self):
        return Label(text='Hello
world')

if __name__ == '__main__':
    MyApp().run()
```

The output for this Hello World app is shown in Figure 4.

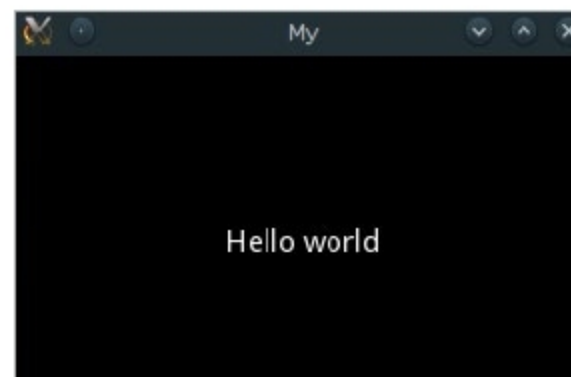


Figure 4: Kivy 'Hello World' app

You can add more controls with the following code:

```
from kivy.app import App
from kivy.uix.gridlayout import
GridLayout
from kivy.uix.label import Label
from kivy.uix.textinput import
TextInput

class LoginScreen(GridLayout):

    def __init__(self, **kwargs):
        super(LoginScreen, self).__
init__(**kwargs)
        self.cols = 2
        self.add_
widget(Label(text='User Name'))
        self.username =
```

```
TextInput(multiline=False)
        self.add_widget(self.username)
        self.add_
widget(Label(text='password'))
        self.password =
TextInput(password=True,
multiline=False)
        self.add_widget(self.password)

class MyApp(App):

    def build(self):
        return LoginScreen()

if __name__ == '__main__':
    MyApp().run()
```

In this code, we have created a grid layout with two columns. Then, text input and label widgets are added for building a login screen.



Figure 5: Kivy text input and label

There is a gallery of applications available, which have been built with Kivy, at <https://kivy.org/doc/stable/examples/gallery.html>.

BeeWare

BeeWare works on the theme 'Write once. Deploy everywhere'. BeeWare enables building apps in Python that can be released on the different platforms/environments listed below:

- Android
- iOS
- Linux
- Windows
- MacOS
- tvOS

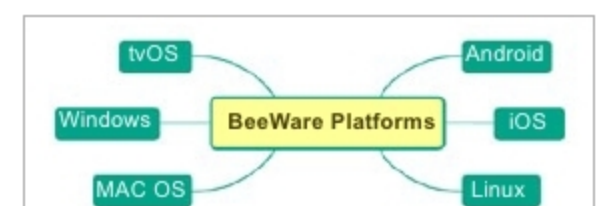


Figure 6: BeeWare platforms